Mixing C and Assembly

CSE 4095

Special Topics in Computer Engineering III:
Introduction to Embedded Systems
Spring 2023

Taken from Dr. Yifeng Zhu's Lecture Notes

Embedded Systems with ARM Cortex-M Microcontrollers in Assembly Language and C – Chapter 10

Motivation

1. An experienced programmer might want to optimize a performance-critical function manually in assembly, instead of relying on compilers.
   - Compilers often have limited intelligence in optimizing these functions.

2. Writing a program in assembly allows a programmer to use processor specific instructions.
   - For example, ROR (rotate right) and RRX (rotate right extended)

3. Assembly programs can directly access hardware, which is especially useful for device drivers and processor booting code

Basic Data Types in C

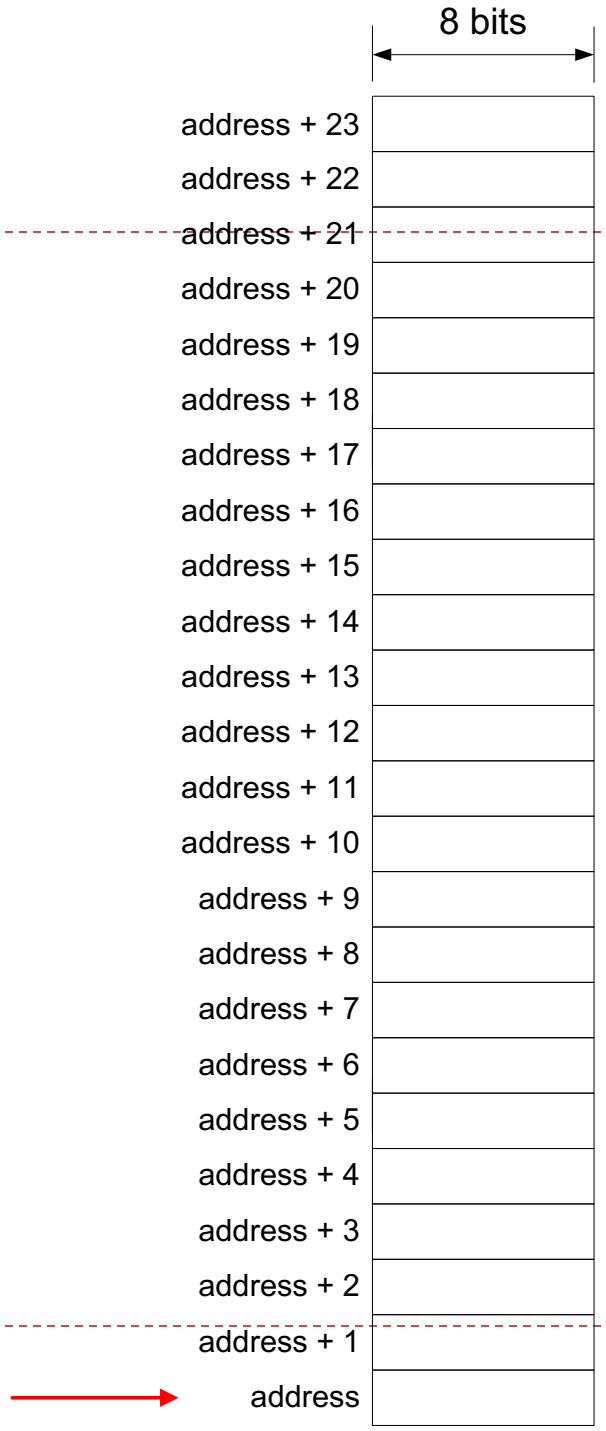
Data Type	Data Size (bits)	Alignment	Data Range
`bool`	8	byte	0 or 1. Bits 1-7 are ignored
`char`	8	byte	-128 - 127(signed) or 0 - 255(unsigned)
`int`	32	word	-2,147,483,648 - 2,147,483,647(signed) or 0 - 4,294,967,296(unsigned)
`short int`	16	halfword	-32,768 - 32,767(signed) or 0 - 65,536(unsigned)
`long int`	32	word	same as int
`long long`	64	word	-9,223,372,036,854,775,808 - 9,223,372,036,854,775,807(signed) or 0 - 18,446,744,073,709,551,616(unsigned)
`float`	32	word	+/- 1.4023×10^{-45} to $3.4028 \times 10^{+38}$, always signed
`double`	64	word	+/- 4.9406×10^{-324} to 1.7977×10^{308}, always signed
`long double`	96	word	very large range

Load and Store Data

unsigned char	**LDRB/STRB**
unsigned short	**LDRH/STRH**
unsigned int	**LDR/STR**
char	**LDRSB/STRB**
short	**LDRSH/STRH**

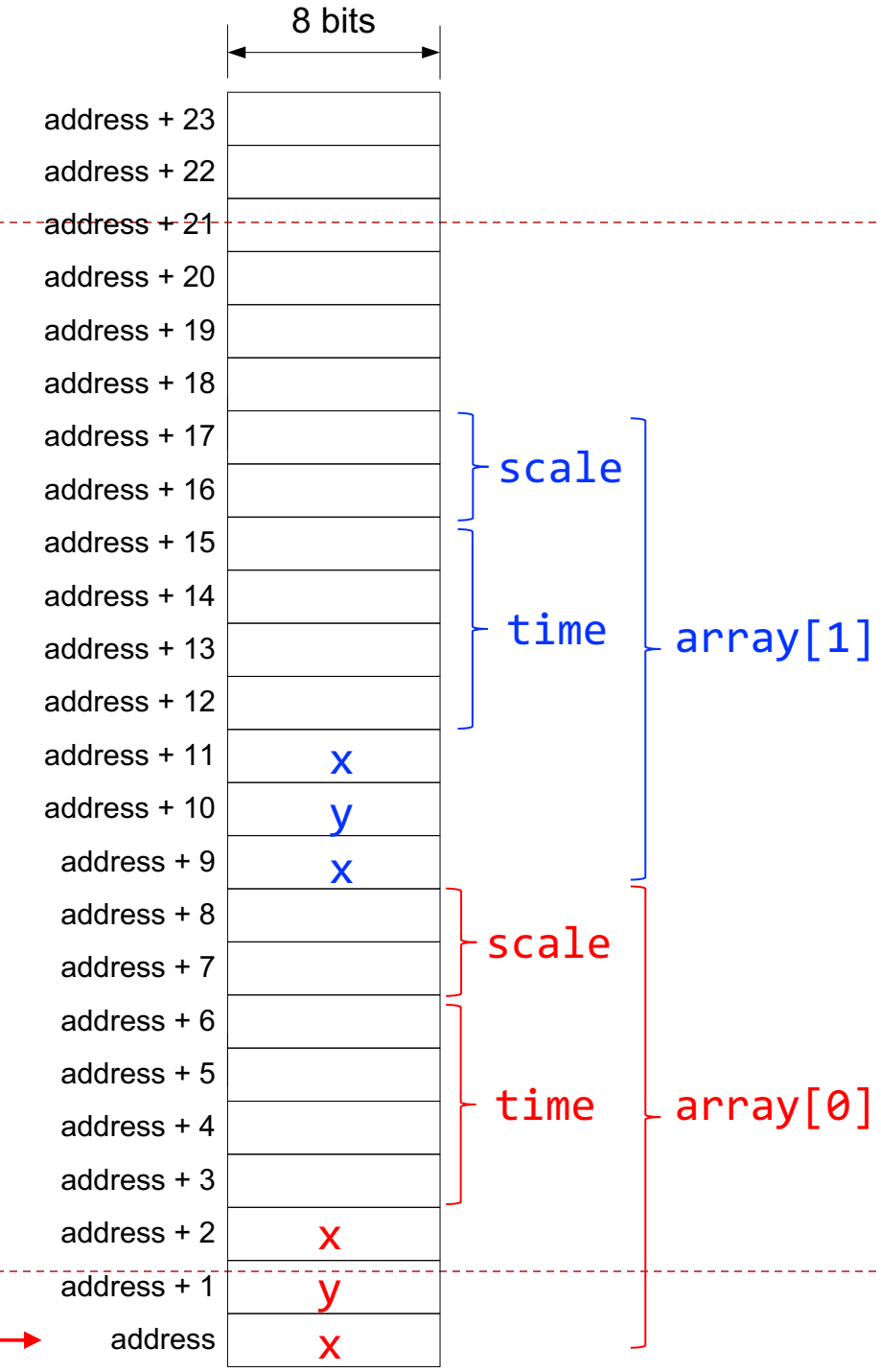
Struct

```
struct Position {
  char x;
  char y;
  char x;
  int time;
  short scale;
} array[10];
```



Struct

```
struct Position {
  char x;
  char y;
  char x;
  int time;
  short scale;
} array[10];
```



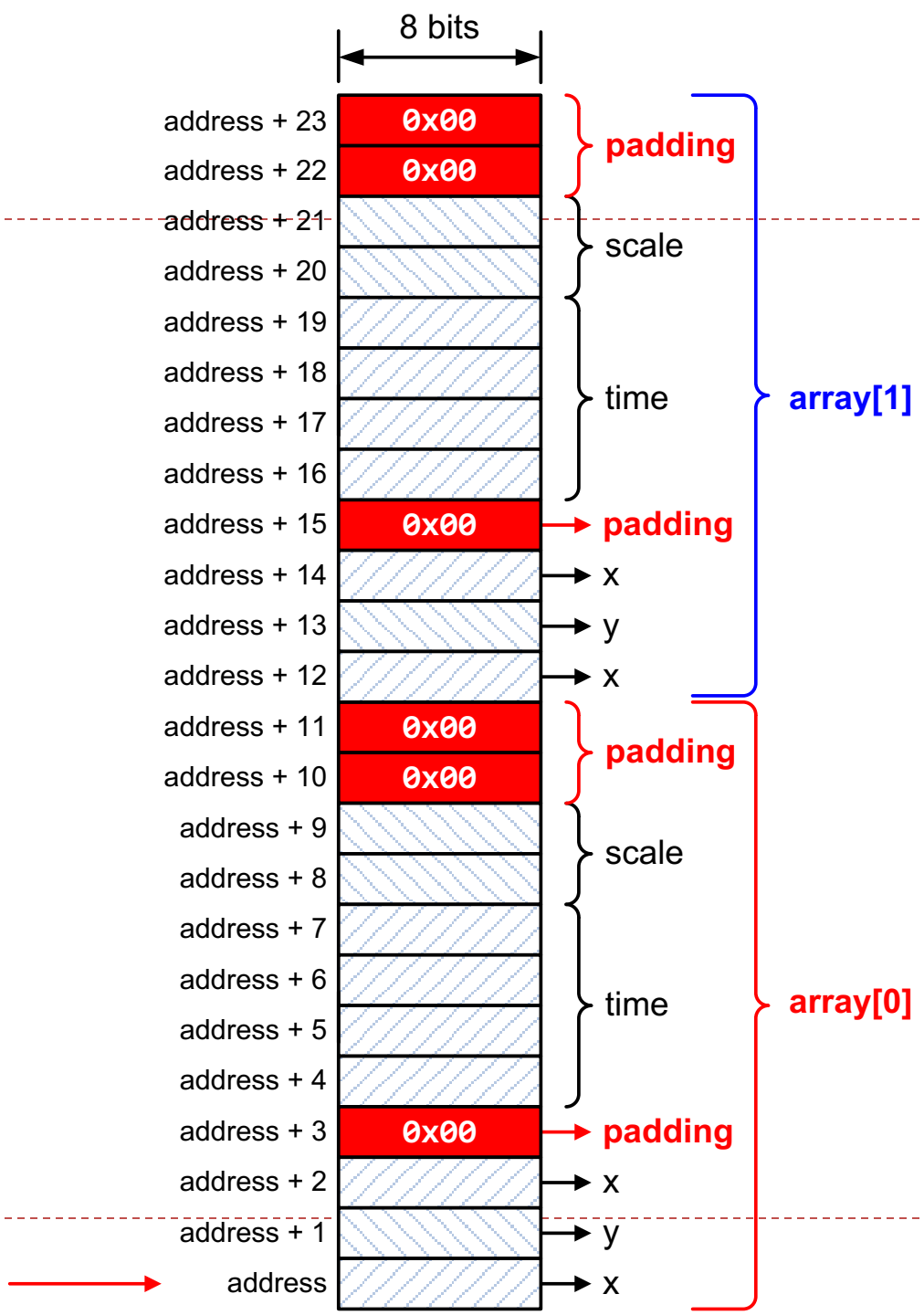
Does the memory layout look like this?

Struct

```
struct Position {
  char x;
  char y;
  char x;
  int time;
  short scale;
} array[10];
```



```
struct Position {
  char x;
  char y;
  char x;
  char padding_1[1];
  int time;
  short scale;
  char padding_2[2];
} array[10];
```



Why is Data Alignment Important?

- **Enforcing data alignment is to improve the memory performance.**
  - A memory system consists of multiple storage units, and the processor typically distributes data among these units in a round-robin fashion.
  - The data alignment ensures that all data of a variable stored in different memory units meet this requirement
  - When the processor reads a properly aligned variable, only one access is required to transfer the data out of these memory units.
  - Otherwise, two separate memory accesses might be necessary, slowing down the processor performance

Loading Unaligned vs. Aligned data

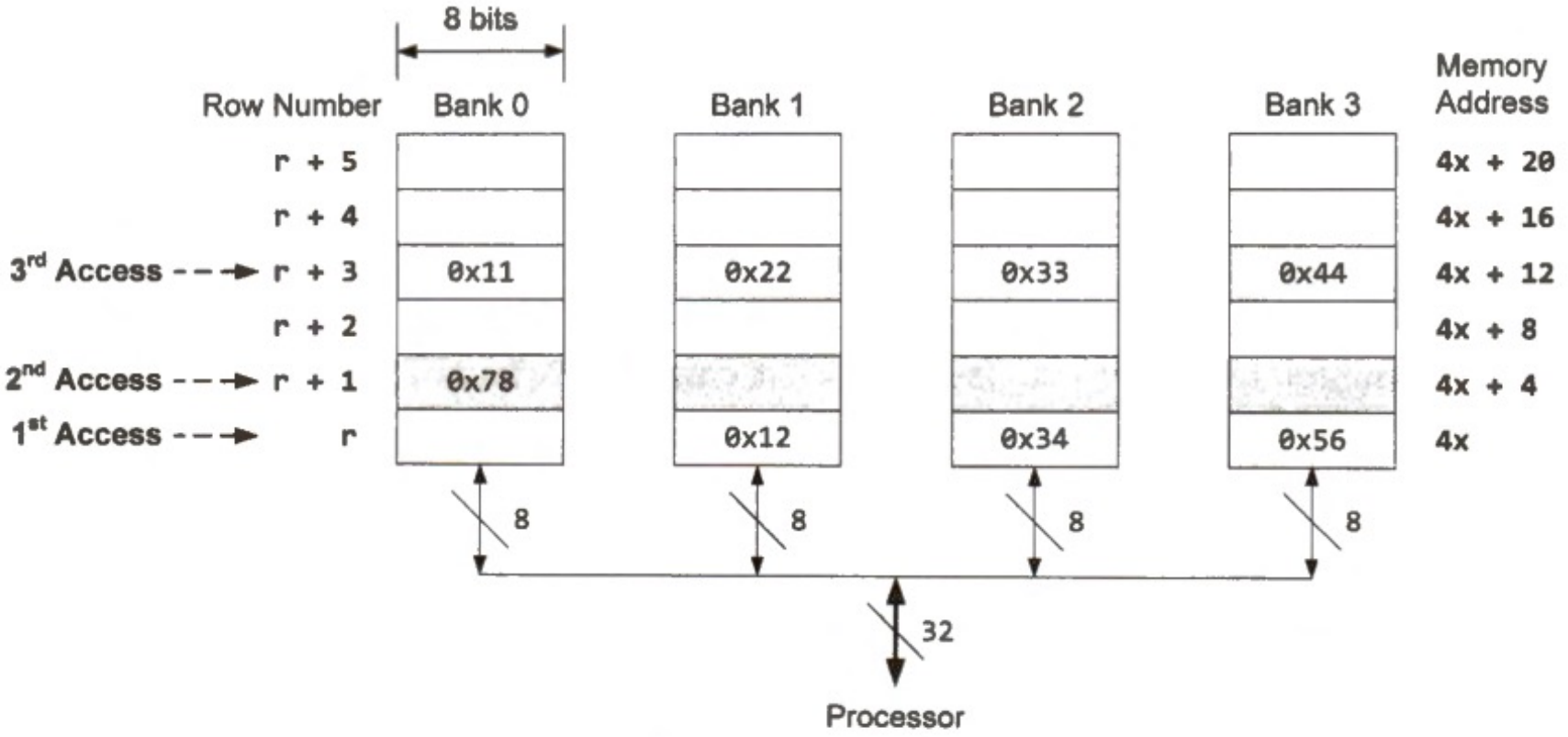


Figure 10-1. Loading unaligned data `0x78563412` takes two accesses even if the processor allows unaligned memory accesses. Loading aligned data `0x44332211` takes only one access.

ALIGN Directive

```
   AREA myData, DATA, ALIGN=2
   ; word aligned

a DCB   0x11

   ALIGN 4
   ; word aligned
b DCD   0x12345678

   ALIGN 4,3
   ; word aligned with an offset of 3
c DCB   0x22

   ALIGN 4,2
   ; word aligned with an offset of 2
d DCW   0xAABB
```

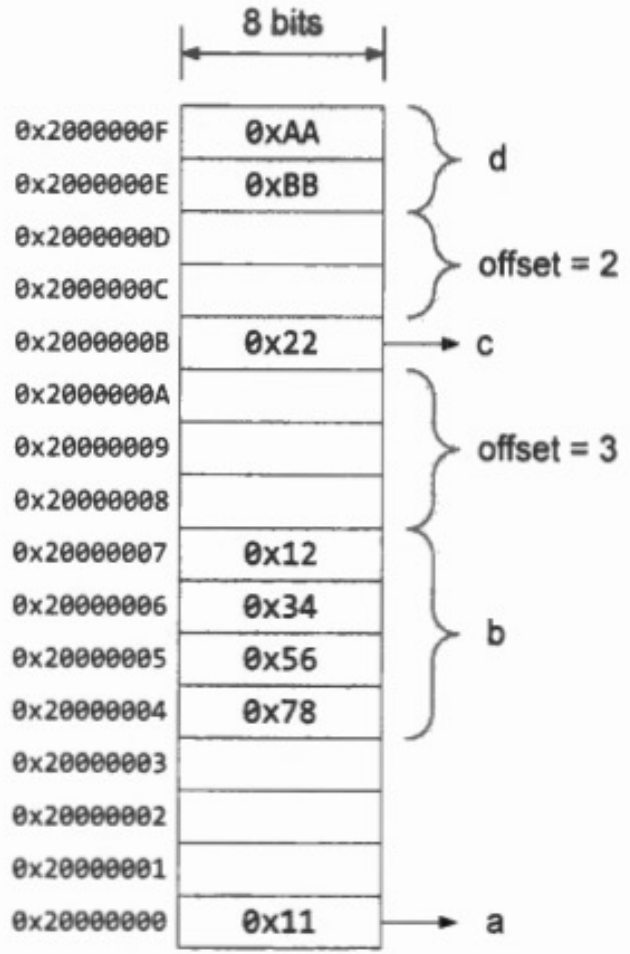
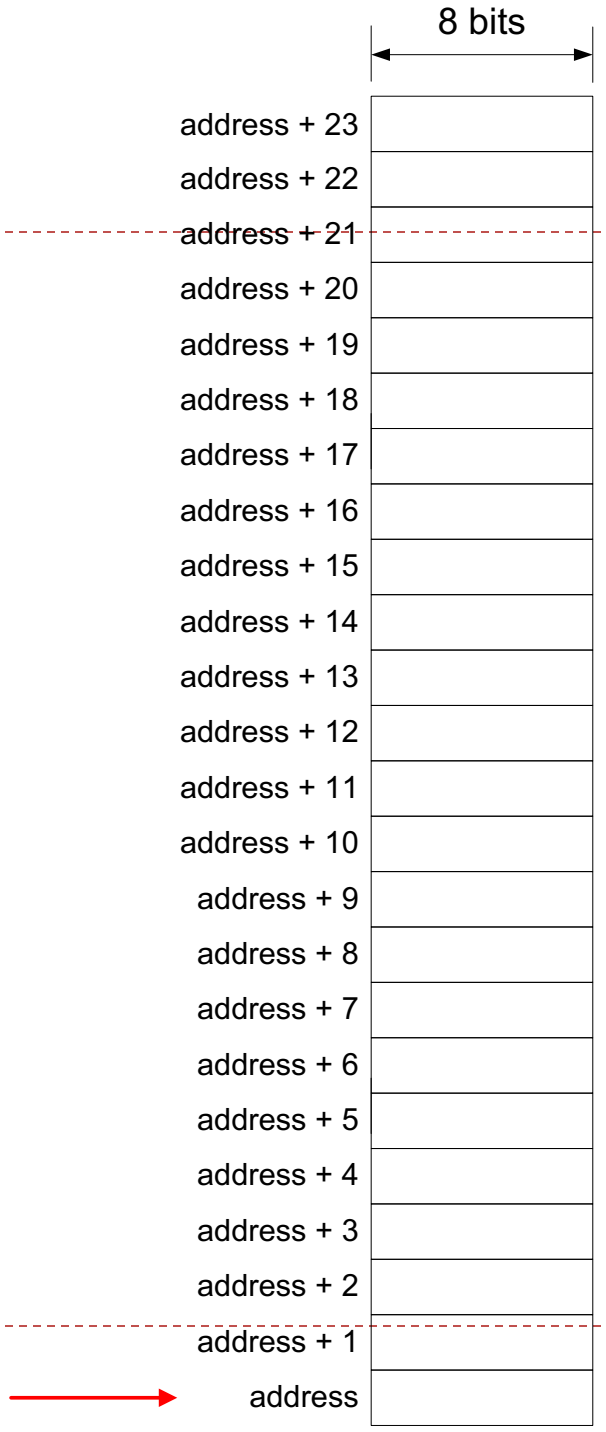


Figure 10-2. Memory layout

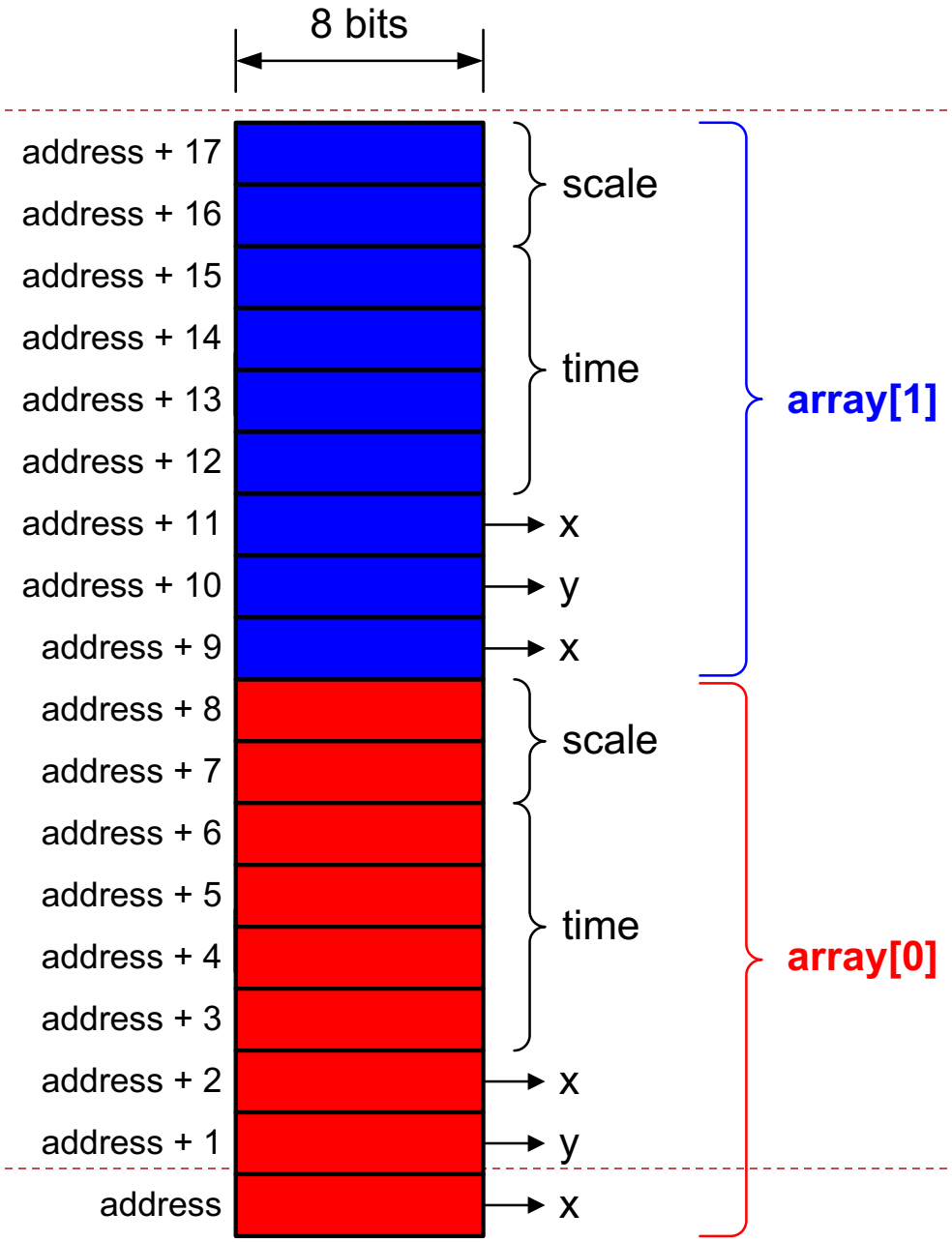
Struct

```
__packed struct Position {
  char x;
  char y;
  char x;
  int time;
  short scale;
} array[10];
```



Struct

```
__packed struct Position {
  char x;
  char y;
  char x;
  int time;
  short scale;
} array[10];
```



Struct

```
struct Position {
  char x;
  char y;
  char x;
  char padding_1[1];
  int time;
  short scale;
  char padding_2[2];
} array[10];
```

Optimized for speed (default)

```
__packed struct Position {
  char x;
  char y;
  char x;
  int time;
  short scale;
} array[10];
```

Optimized for space

Special Variables: 1. Static Variables

C Code

```
int foo();

int main(void) {
   int y;
   y = foo();  // y = ?
   y = foo();  // y = ?
   y = foo();  // y = ?
   while(1);
}

int foo() {
   int x = 5;
   x = x + 1;
   return(x)
}
```

C Code

```
int foo();

int main(void) {
   int y;
   y = foo();  // y = ?
   y = foo();  // y = ?
   y = foo();  // y = ?
   while(1);
}

int foo() {
   static int x = 5;
   x = x + 1;
   return(x)
}
```

Static Variables

C Code	Assembly Code

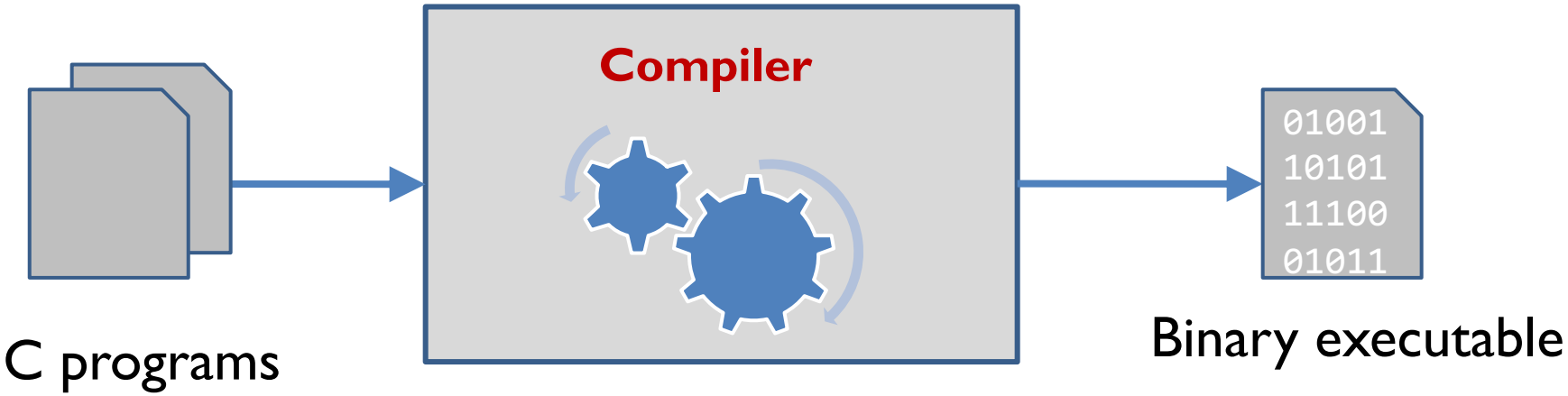
`int foo();`
 	
`int main(void) {`
`  int y;`
`  y = foo();  // y = 6`
`  y = foo();  // y = 6`
`  y = foo();  // y = 6`
`  while(1);`
`}`
 	
`int foo() {`
`  int x = 5;  // local variable`
`  x = x + 1;`
`  return(x)`
`}`
	`  AREA static_demo, CODE`
	`  EXPORT __main`
	`  ALIGN`
	`  ENTRY`
	`__main  PROC`
	`  BL  foo  ; r0 = 6`
	`  BL  foo  ; r0 = 6`
	`  BL  foo  ; r0 = 6`
	`stop  B  stop`
	`  ENDP`
	
	`foo  PROC`
	`  MOV  r0,#5`
	`  ADD  r0,r0,#1`
	`  BX  LR`
	`  ENDP`
	`  END`

Static Variables

C Code	Assembly Code

`int foo();`
 	
`int main(void) {`
`  int y;`
`  y = foo();  // y = 6`
`  y = foo();  // y = 7`
`  y = foo();  // y = 8`
`  while(1);`
`}`
 	
`int foo() {`
`  // x is initialized only once`
`  static int x = 5; // a static variable`
`  x = x + 1;`
`  return(x)`
`}`
	`  AREA static_demo, CODE`
	`  EXPORT __main`
	`  ALIGN`
	`  ENTRY`
	`__main  PROC`
	`  BL  foo  ; r0 = 5`
	`  BL  foo  ; r0 = 6`
	`  BL  foo  ; r0 = 7`
	`stop  B  stop`
	`  ENDP`
	
	`foo  PROC`
	`  LDR  r1,=x`
	`  LDR  r0,[r1]`
	`  ADD  r0,r0,#1`
	`  STR  r0,[r1]`
	`  BX  LR`
	`  ENDP`
	
	`  AREA myData, DATA`
	`  ALIGN`
	`x  DCD  5`
	`  END`

Compiler



Performance, Performance, Performance!

- Compiler can make aggressive optimization
  - Cache results in register to reduce memory accesses
  - Reorder computations
  - Eliminate useless and redundant computations

Volatile Variables

- When the compiler optimizes a C program, a hard-to-find hidden error is that the program mistakenly reuses the value of a variable stored in a register, instead of reloading it from memory each time.

- To avoid such compilation error, the program should declare the variable as **volatile**

- The keyword **volatile** forces the compiler
  - to load the variable value from the memory whenever this variable is read, and
  - always stores the variable in memory whenever it is written.

```c
int flag = 0;

void SysTick_Handler(void){
    flag = 1;
    // Turn on the green LED
    GPIOE->ODR |= GPIO_ODR_ODR_8;
}

int main(void){
    GPIO_init();
    SysTick_init();

    while(flag == 0);

    // Turn on the red LED
    GPIOB->ODR |= GPIO_ODR_ODR_2;

    while(1);
}
```

Without optimization

```
        ...
        LDR r0, =flag   ; Get memory address
loop    LDR r1, [r0]    ; Read memory
        CMP r1, #0
        BEQ loop
        ...
```

With optimization

```
        ...
        LDR r0, =flag   ; Get memory address
        LDR r1, [r0]    ; Read memory
loop    CMP r1, #0
        BEQ loop
        ...
```

A dead loop since flag was 0. Thus, the program is blocked here and failed to turn on the red LED.

Solution

```
volatile int flag = 0;

void SysTick_Handler(void){
    flag = 1;
    // Turn on the green LED
    GPIOE->ODR |= GPIO_ODR_ODR_8;
}

int main(void){
    GPIO_init();
    SysTick_init();

    while(flag == 0);

    // Turn on the red LED
    GPIOB->ODR |= GPIO_ODR_ODR_2;

    while(1);
}
```

With/without optimization

```
        ...
        LDR r0, =flag  ; Get memory address
loop    LDR r1, [r0]   ; Read memory
        CMP r1, #0
        BEQ loop
        ...
```

Volatile variables

- Qualifier `volatile` tells the compiler that this value may change at any time
  - Do not apply any optimizations to remove reads or writes to these variables

Example:

```
volatile int x;
y = x + x + x + x;
```

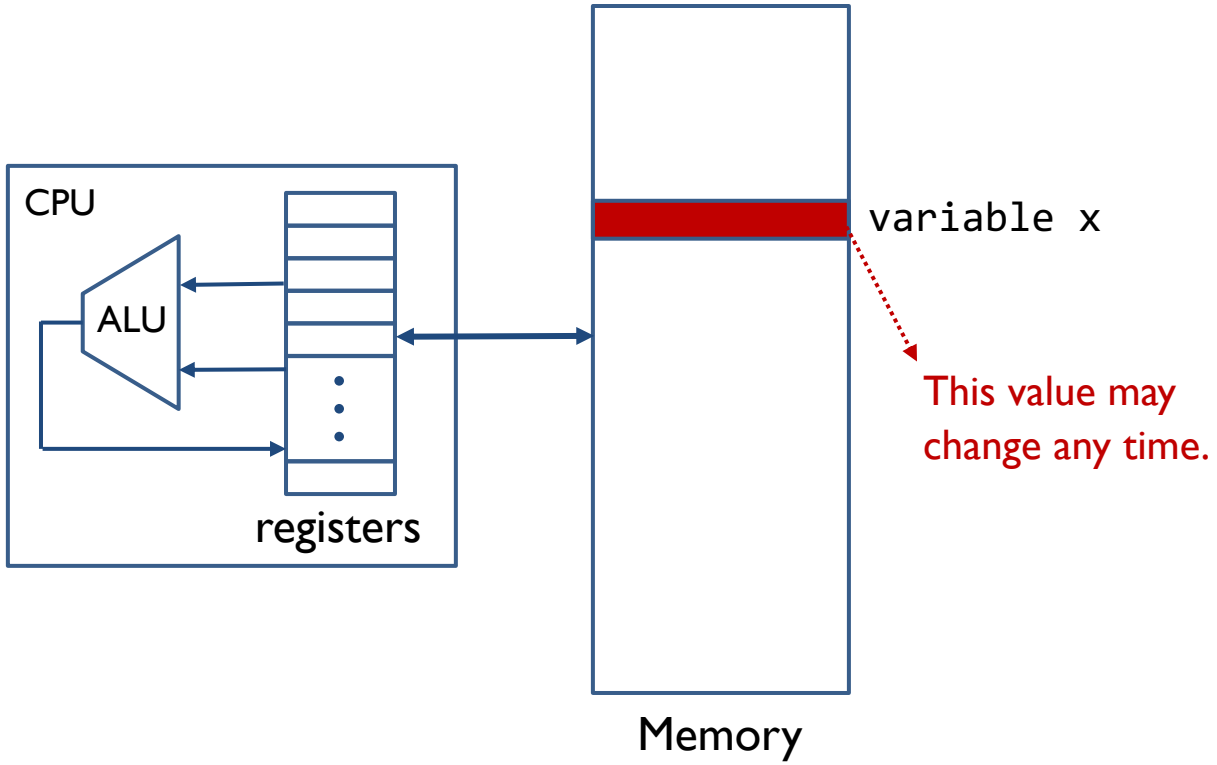
The computation cannot complete within a single clock cycle and x can change any time.

Compiler cannot optimize it as:

```
y = 4 * x;
```

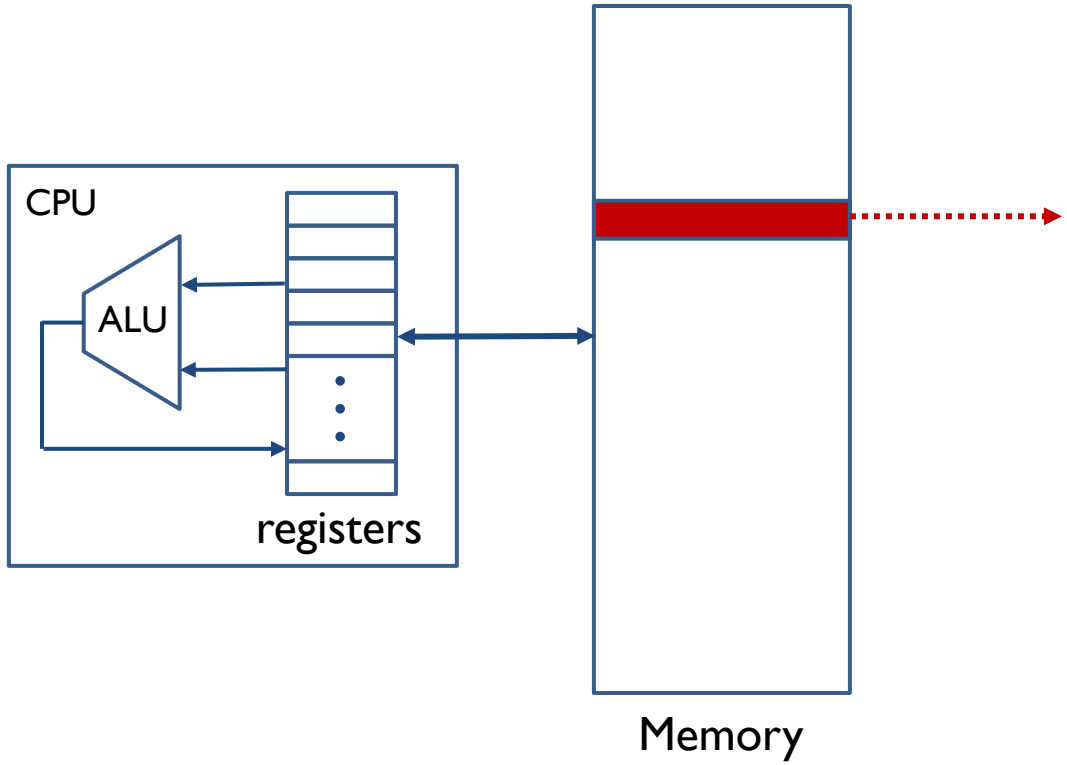
or

```
y = x << 2;
```



Volatile variables

- Qualifier `volatile` tells the compiler that this value may change at any time
  - Do not apply any optimizations to remove reads or writes to these variables



A memory value may be changed anytime by hardware.

Input data registers:

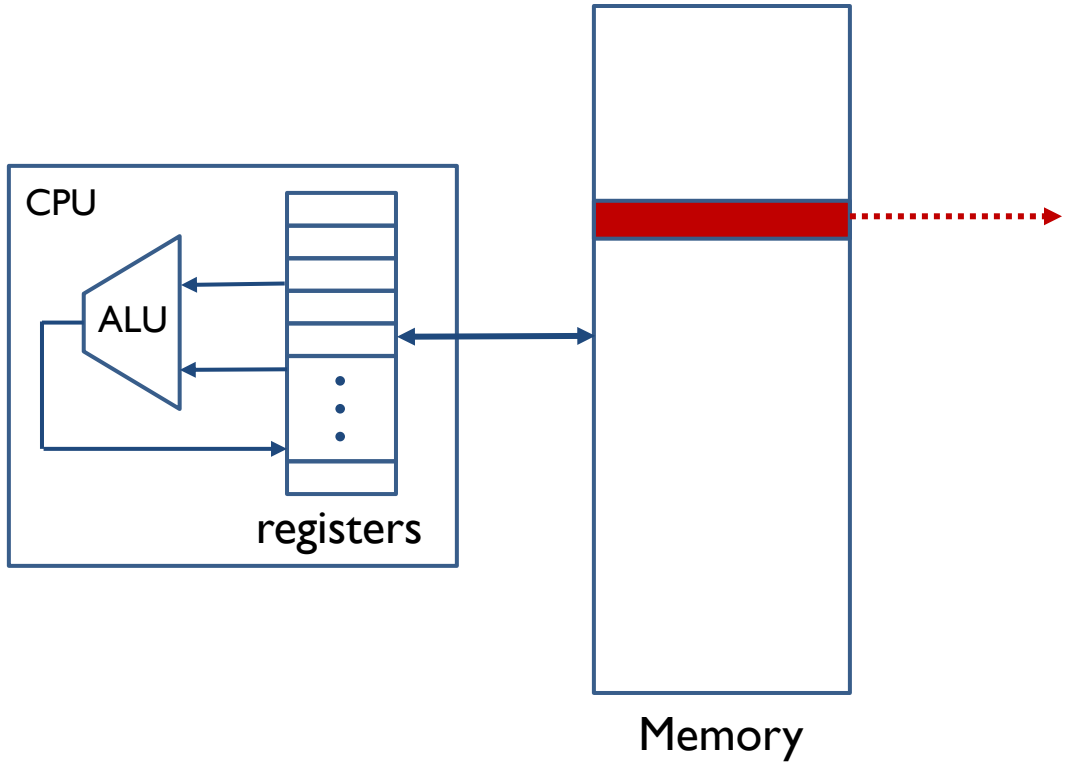
- `GPIOB->IDR`
- `SPI1->DR`
- `USART1->DR`

Status registers:

- `USART1->SR`

Volatile variables

- Qualifier `volatile` tells the compiler that this value may change at any time
  - Do not apply any optimizations to remove reads or writes to these variables



This value may be changed anytime by software.

- Interrupt handlers
- Another thread (with OS or RTOS)

Three Common Usages of volatile

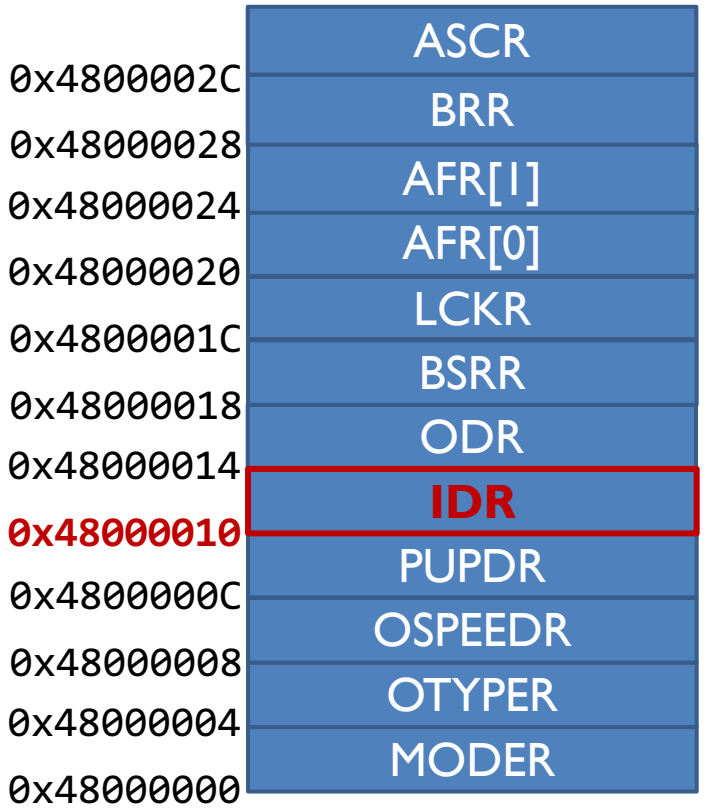
- Use volatile on accessing memory-mapped I/O registers

```
#define GPIOA_IDR (*((uint32_t volatile *) 0x48000010))

void main(void){
    ...

    // Read the input on pin 2
    Pin2 = GPIOA_IDR >> 2;

    ...
}
```



GPIOA Peripheral Registers

Three Common Usages of volatile

- Use volatile on accessing memory-mapped I/O registers
- Use volatile on global variables accessed in interrupt service routines
- Use volatile on global variables shared between multi-thread tasks

```
volatile uint32_t counter;

void SysTick_Handler(void){
    counter++;
}
```

Syntax of volatile keyword

- Declare a variable volatile

```
volatile int x;
int volatile y;
```

Pointers

- A pointer to an int

```
int* pi;
```

- A pointer to a volatile int

```
int volatile* pvi;
```

- A volatile pointer to an int

```
int* volatile vpi;
```

- A volatile pointer to a volatile int

```
int volatile* volatile vpvi;
```

Mixing C and Assembly

- Inline assembly:
  - Using assembly instructions embedded in a C program

- C and assembly in different files:
  - C program
    - Invokes assembly functions
    - Accesses assembly variables
  - Assembly program
    - Invokes C functions
    - Accesses C variables

Inline Assembly

- A block of assembly code, called *inline assembly*, can be directly embedded in a C program.
- It is convenient for programmers because it does not require different assemble and link processes.
- Another advantage of inline assembly is that it can flexibly access C variables without export and import operations, which would be needed if the assembly code were written as an assembly subroutine.

Assembly Function in a C Program

```
__asm int sum4(int a, int b, int c, int d){
        // arguments stored in r0-r3
        PUSH {r4, lr}
        MOV r4, r0          ; r0 = 1st argument
        ADD r4, r4, r1      ; r1 = 2nd argument
        ADD r4, r4, r2      ; r2 = 3rd argument
        ADD r0, r4, r3      ; r3 = 4th argument, r0 = return
        POP {r4, pc}
}

int main(void){
        int s = sum4(1, 2, 3, 4);
        while(1);
}
```

Assembly Function in a C program

- Accessing special instructions

```
__asm uint8_t __clz(uint32_t a){
        // arguments stored in r0
        CLZ r0, r0 ; r0 = 1st argument, CLZ = Counting Leading Zeros
        BX  LR     ; return result in r0
}

int main(void){
        int s = __clz(0xFF);
        while(1);
}
```

Inline Assembly Instructions in a C Program

```
int sum4(int a, int b, int c, int d){
	int t;
	__asm {
		ADD t, a, b;  // t, a, and b are virtual registers
		ADD t, c;     // Cannot directly access r0-r15
		ADD t, d;     // Use C comment style
	}
	return t;
}

int main(void){
	int s = sum4(1, 2, 3, 4);
	while(1);
}
```

C Calling Assembly

C Program (main.c)

```
char str[25] = "Hello!";

extern int strlen(char* s);

int main(void){
    int i;
    i = strlen(str);
    while(1);
}
```

Assembly Program (strlen.s)

```
        AREA stringLength, CODE
        EXPORT strlen       ; make strlen visible
        ALIGN
strlen  PROC
        PUSH {r4, lr}       ; preserve r4 and lr
        MOV  r4, #0         ; initialize length
loop    LDRB r1, [r0, r4]   ; r0 = string address
        CBZ  r1, exit       ; branch if zero
        ADD  r4, r4, #1     ; length++
        B    loop           ; do it again
exit    MOV  r0, r4         ; place result in r0
        POP  {r4, pc}       ; exit
        ENDP
```

Accessing C Variables

C Program (main.c)

```
int counter;

extern int getValue();
extern void setValue(int c);

void increment();

int main(void) {
   int c = 0;
   setValue(1);
   increment();
   c = getValue();
   while(1);
}

void increment(){
   counter += 2;
}
```

Assembly Program (count.s)

```
            AREA count, CODE
            IMPORT counter
            ALIGN
setValue    PROC
            EXPORT setValue
            LDR r1, =counter
            STR r0, [r1]
            BX  lr
            ENDP
getValue    PROC
            EXPORT getValue
            LDR r1, =counter
            LDR r0, [r1]
            BX  lr
            ENDP
increment   PROC
            EXPORT increment [WEAK]
            LDR r1, =counter
            LDR r0, [r1]
            ADD r0, r0, #1
            STR r0, [r1]
            BX  lr
            ENDP
            END
```

Assembly Calling C

Assembly Program (main.s)

```
        AREA my_strlen, CODE
        EXPORT __main
        IMPORT strlen
        ALIGN
        ENTRY

__main  PROC
        LDR   r0, =str
        BL    strlen
stop    B     stop
        ENDP

        AREA myData, DATA
        ALIGN
str     DCB   "12345678",0
        END
```

C Program (strlen.c)

```
int strlen(char *s){
   int i = 0;

   while( s[i] != '\0')
      i++;

   return i;
}
```

Accessing Data Defined in Assembly

Assembly Program	C Program

AREA main, CODE
EXPORT __main
IMPORT getValue
IMPORT increment
IMPORT setValue
ALIGN
ENTRY
 	
__main MOVS r0,#1
BL setValue
BL increment
BL getValue
stop B stop
 	
AREA myData, DATA
EXPORT counter
counter DCD 0
END
	extern int counter;
	
	int getValue() {
	return counter;
	}
	
	void increment() {
	counter++;
	}
	
	void setValue(int c) {
	counter = c;
	}

Exercise

- Write a subroutine *max4* in assembly to find the maximum value among four signed integers.
- These integers are passed to the subroutine via registers.
- Write a C program to test the *max4* subroutine.

Exercise: Solution

```
// main.c
extern int max4(int a, int b, int c, int d);

int main() {
        volatile int m;
        m = max4(10, 11, 15, 12);
        while(1);
}
```

```
        ; max4.s
        AREA myCode,CODE
        EXPORT max4

max4  PROC
        CMP r0, r1
        MOVLT r0, r1
        CMP r0, r2
        MOVLT r0, r2
        CMP r0, r3
        MOVLT r0, r3
        BX  lr
        ENDP
        END
```